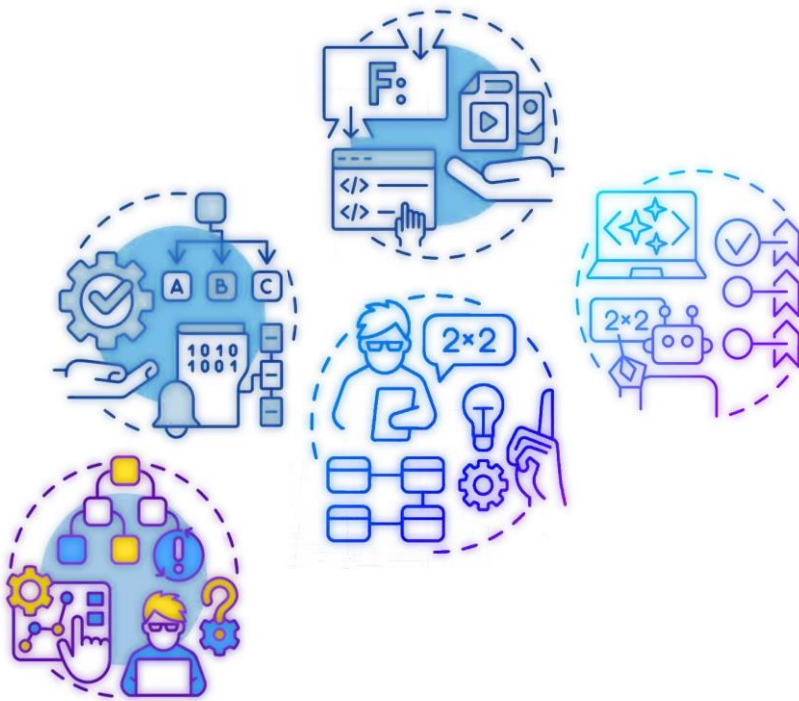




Paradigma Funcional

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. J. Exequiel Benavidez
Ing. Jimena Bourlot

Tecnologías de la Programación

1 – Expresiones Let

La expresión `let` en Racket no crea una variable en el sentido imperativo. En realidad, `let` introduce un nuevo enlace entre un nombre y un valor. Este enlace es válido solo dentro del cuerpo de la expresión `let`.

La forma sintáctica `Let` incluye pares de variables-expresiones junto con una secuencia de expresiones que representan el cuerpo del `Let`

```
(let ((var1 val) [(var2 val)...]) exp1 exp2 ...)
```

1 – Expresiones Let

Puntos clave:

1. **Inmutabilidad:** El valor al que se enlaza un nombre con let no puede ser modificado. Si se intenta reasignar un valor a ese nombre dentro del mismo ámbito de let, no se está cambiando el valor original, sino que se está creando un nuevo enlace (lo cual no está permitido directamente dentro del mismo let).
2. **Ámbito Léxico:** El enlace creado con let tiene un ámbito léxico. Esto significa que el nombre solo es visible y accesible dentro del bloque de código definido por el let.
3. **No hay mutación, hay enlaces:** En lugar de pensar en let como una forma de crear variables mutables, es más preciso verlo como una forma de crear nombres para valores inmutables dentro de un ámbito limitado.

1 – Expresiones Let

Ejemplo en Racket

```
(let ([x 10])  
  (let ([x 20])  
    (+ x 5))  
  x)
```

Aquí, hay dos enlaces para el nombre x. El x del let exterior está enlazado a 10. El x del let interior está enlazado a 20. Son dos enlaces distintos. Dentro del let interior, x se refiere a 20. Fuera del let interior, x se refiere a 10. El valor 10 nunca cambió. El valor 20 nunca cambió. Simplemente, el símbolo x se asoció con diferentes valores en diferentes ámbitos.

1 – Expresiones Let

Beneficios de este enfoque

1. **Predecibilidad:** El comportamiento de un programa se vuelve más fácil de razonar, ya que los valores no cambian inesperadamente.
2. **Transparencia referencial:** Las expresiones siempre producen el mismo resultado dado los mismos argumentos, lo que facilita la depuración y las pruebas.
3. **Paralelismo:** La inmutabilidad facilita la ejecución concurrente de programas, ya que no hay necesidad de preocuparse por la sincronización de accesos a datos mutables.

En resumen, aunque let pueda parecer que introduce variables, en realidad introduce enlaces inmutables con ámbito léxico. Este es un concepto fundamental en la programación funcional.

1 – Expresiones Let

También se usa para simplificar expresiones

- $(+ (* 4 4) (* 4 4)) \Rightarrow 32$
- $(\text{let } ((a (* 4 4))) (+ a a)) \Rightarrow 32$

1 – Expresiones Let

Es posible anidar Lets

- (let ((a 4) (b -3))
 (let ((a-squared (* a a))
 (b-squared (* b b)))
 (+ a-squared b-squared))) → 25
- (let ((x 1))
 (let ((x (+ x 1)))
 (display x))) → 2

1 – Expresiones let-bound

El término "let-bound" se refiere a una variable que ha sido introducida por una expresión **let**.

Una variable es "let-bound", significa que esa variable ha sido ligada (o asociada) a un valor mediante una expresión **let**. En otras palabras, la expresión **let** crea un nuevo ámbito léxico en el que esas variables tienen un significado y un valor específicos.

```
(let ([x 5]
      [y 10])
  (+ x y))
```

En este ejemplo, las variables **x** e **y** son "let-bound". Dentro del cuerpo del **let**, **x** está ligada al valor 5, e **y** está ligada al valor 10. Estas ligaduras sólo son válidas dentro del ámbito del **let**

1 – Expresiones let-bound

El concepto de "let-bound" es importante en programación funcional por las siguientes razones:

- **Inmutabilidad:** En un lenguaje funcional como Racket, las variables let-bound son inmutables. Esto significa que una vez que una variable es ligada a un valor, esa ligadura no puede cambiar dentro de su ámbito.
- **Ámbito léxico:** Las variables let-bound tienen ámbito léxico, lo que significa que su visibilidad y valor están determinados por la estructura del código, y no por el orden de ejecución. Esto hace que el código sea más predecible y fácil de razonar.
- **Evitar efectos secundarios:** Al limitar el ámbito de las variables y garantizar su inmutabilidad, **let** ayuda a prevenir efectos secundarios y a escribir código más limpio y modular.

2 - Definición de variables y funciones

En Racket, la forma principal de definir valores y funciones es mediante la palabra clave **define**.

1. De un valor:

(define nombre valor)

Esto asocia el **valor** al **nombre**. A partir de este momento, cada vez que se utilice **nombre**, se recuperará el **valor** al que está asociado.

```
(define pi 3.14159)
```

```
(define mensaje "Hola, mundo")
```

2 - Definición de variables y funciones

En Racket, la forma principal de definir valores y funciones es mediante la palabra clave **define**.

2. De una función:

(define (nombre parametro1 parametro2 ...) cuerpo-de-la-funcion)

Esto define una función llamada **nombre** que toma uno o más **parámetro1**, **parametro2**, etc., y cuyo comportamiento se describe en el **cuerpo-de-la-funcion**.

(define (sumar x y)
 (+ x y))

(sumar 5 3) ; Devuelve 8
(sumar 10 -2) ; Devuelve 8

2 - Definición de variables y funciones

Puntos importantes sobre **define**:

- **Ámbito de Visibilidad:** Las definiciones realizadas con **define** a nivel superior (fuera de cualquier otra expresión) son visibles en todo el programa. Dentro de una función, las definiciones con **define** son locales a esa función.
- **Evaluación:** Cuando se define un valor con **define**, la **valor** se evalúa antes de realizar la asociación con el **nombre**.
- **Inmutabilidad:** En Racket, los valores asociados a los nombres mediante **define** son inmutables. Esto significa que, una vez que un nombre está asociado a un valor, esa asociación no cambia. Sin embargo, se puede crear una nueva asociación con el mismo nombre en un ámbito más interno, "shadowing" la anterior.

se refiere a lo que una variable declarada en un ámbito local tiene el mismo nombre que otra variable declarada en un ámbito más externo. La variable local "eclipsa" o "esconde" a la variable del ámbito externo, impidiendo el acceso a esta última dentro del ámbito de la variable local

3 - Funciones anónimas (también conocidas como lambdas)

Una función anónima es una función que no está ligada a un nombre. A diferencia de las funciones que se definen con un nombre (como con `def` en Python o `define` en Racket), las funciones anónimas se definen en el lugar donde se van a utilizar.

¿Por qué son útiles?

- Facilitan el tratamiento de las funciones como valores.
- Son la forma natural de definir argumentos para funciones de orden superior.
- Mejoran la expresividad y concisión del código.
- Apoyan la composición de funciones.
- Ayudan a mantener limpio el espacio de nombres.

3 - Funciones anónimas (también conocidas como lambdas)

En Racket, las funciones anónimas se crean con la palabra clave lambda.

(lambda (x) (* x x))

((lambda (x) (* x x)) 5) ; Aplica la función anónima a 5, devuelve 25

las funciones anónimas son una herramienta poderosa en la programación funcional que permite definir funciones concisas en el lugar donde se necesitan, especialmente cuando se trabajan con funciones de orden superior o funciones simples de un solo uso.

4 - Control de flujo funcional - Condicionales

En Racket las condicionales se utilizan para ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

Condicional if

La forma más básica de una condicional en Racket es la expresión if, que tiene la siguiente sintaxis:

```
(if condición  
    entonces-expresión  
    si-no-expresión)
```

condición: Es una expresión que se evalúa como verdadera (**#t**) o falsa (**#f**).

entonces-expresión: Es la expresión que se ejecuta si la **condición** es verdadera.

si-no-expresión: Es la expresión que se ejecuta si la **condición** es falsa.

4 - Control de flujo funcional - Condicionales

Ejemplo Condicional if

(define (valor-absoluto x)

(if (< x 0)

(- x) ; Si x es negativo, devuelve su negativo

x)) ; Si x es positivo o cero, devuelve x

4 - Control de flujo funcional - Condicionales

Condicional cond

La expresión cond proporciona una forma más general de expresar condicionales múltiples. Su sintaxis es la siguiente:

```
(cond
  [(condición1) (expresión1)]
  [(condición2) (expresión2)]
  ...
  [else (expresión-por-defecto)])
```

Cada cláusula `[(condición) (expresión)]` se evalúa secuencialmente. Si una `condición` es verdadera, se evalúa su correspondiente `expresión` y se devuelve su valor. No se evalúan las cláusulas restantes.

La cláusula `[else (expresión-por-defecto)]` es opcional. Si ninguna de las condiciones anteriores es verdadera, se evalúa esta expresión.

4 - Control de flujo funcional - Condicionales

Ejemplo Condicional cond

(define (signo x)

(cond

[($< x 0$) -1]

[($= x 0$) 0]

[($> x 0$) 1]))

4 - Control de flujo funcional - Condicionales

Diferencias clave

- `if` es para condicionales simples con dos opciones (verdadero o falso)
- `cond` es para condicionales múltiples con varias opciones.

En resumen, tanto `if` como `cond` son herramientas esenciales para controlar el flujo de ejecución de un programa en Racket, permitiendo que el código se adapte a diferentes situaciones.

4 - Operadores lógicos - not

En Racket, los operadores lógicos se utilizan para realizar operaciones booleanas.

1. **not**

La función **not** toma un valor como argumento y devuelve su negación lógica.

- Si el argumento es verdadero (`#t`), `not` devuelve falso (`#f`).
- Si el argumento es falso (`#f`), `not` devuelve verdadero (`#t`).
- En Racket, cualquier valor que no sea `#f` se considera verdadero.

Ejemplos:

- `(not #t) → #f`
- `(not "false") → #f` (porque `"false"` no es `#f`)
- `(not #f) → #t`

4 - Operadores lógicos - or

2. **or**

La función **or** toma uno o más valores como argumentos y devuelve el primer valor que sea verdadero. Si todos los valores son falsos, devuelve falso.

- **(or)** → **#f** (sin argumentos, se considera falso)
- **(or #t #f)** → **#t** (el primero es verdadero)
- **(or #f #t)** → **#t** (el segundo es verdadero)
- **(or #f #f)** → **#f** (ambos son falsos)

4 - Operadores lógicos - and

3. and

La función **and** toma uno o más valores como argumentos y devuelve el primer valor que sea falso. Si todos los valores son verdaderos, devuelve el último valor (que será verdadero).

- **(and)** -> **#t**
- **(and #t #f)** → **#f** (el segundo es falso)
- **(and #t #t)** → **#t** (ambos son verdaderos)
- **(and #f #t)** → **#f** (el primero es falso)

5 - Predicados

Un predicado es una función que toma uno o más valores como entrada y devuelve un valor de verdad, indicando si una cierta propiedad se cumple o no. En Racket, los valores de verdad son `#t` (verdadero) y `#f` (falso).

En el caso de estos predicados, responden a preguntas sobre la relación entre números:

- `=:` Responde a la pregunta "¿son estos números iguales?".
- `<:` Responde a la pregunta "¿es el primer número menor que el segundo?".
- `>:` Responde a la pregunta "¿es el primer número mayor que el segundo?".
- `<=:` Responde a la pregunta "¿es el primer número menor o igual que el segundo?".
- `>=:` Responde a la pregunta "¿es el primer número mayor o igual que el segundo?".

5 - Predicados

- $(= 5 5)$; $\rightarrow \#t$ (verdadero, 5 es igual a 5)
- $(= 5 6)$; $\rightarrow \#f$ (falso, 5 no es igual a 6)
- $(< 3 7)$; $\rightarrow \#t$ (verdadero, 3 es menor que 7)
- $(< 7 3)$; $\rightarrow \#f$ (falso, 7 no es menor que 3)
- $(> 8 2)$; $\rightarrow \#t$ (verdadero, 8 es mayor que 2)
- $(> 2 8)$; $\rightarrow \#f$ (falso, 2 no es mayor que 8)
- $(<= 4 4)$; $\rightarrow \#t$ (verdadero, 4 es menor o igual que 4)
- $(<= 4 5)$; $\rightarrow \#t$ (verdadero, 4 es menor o igual que 5)
- $(<= 5 4)$; $\rightarrow \#f$ (falso, 5 no es menor o igual que 4)
- $(>= 6 6)$; $\rightarrow \#t$ (verdadero, 6 es mayor o igual que 6)
- $(>= 6 5)$; $\rightarrow \#t$ (verdadero, 6 es mayor o igual que 5)
- $(>= 5 6)$; $\rightarrow \#f$ (falso, 5 no es mayor o igual que 6)

5 - Predicados

Importancia en la programación funcional

Estos predicados son esenciales en la programación funcional porque se utilizan para tomar decisiones y controlar el flujo del programa sin recurrir a efectos secundarios. Por ejemplo, se utilizan en:

- Condicionales (`if`, `cond`) para determinar qué rama de código ejecutar.
- Funciones de filtrado (`filter`) para seleccionar elementos de una lista que cumplan con una cierta condición.
- Funciones de ordenamiento (`sort`) para comparar elementos y determinar su orden.

5 - Predicados

En Racket, muchos predicados (funciones que devuelven `#t` o `#f`) tienen nombres que terminan en un signo de interrogación `?`. Esta es una convención de estilo, no una regla del lenguaje, y ayuda a los programadores a identificar rápidamente qué funciones son predicados.

Predicados que terminan en `?`

`null?`, `eqv?`, `pair?`, `symbol?`, `number?`, y `string?`, siguen esta convención. Indican que estas funciones hacen una pregunta sobre su argumento y devuelven un valor de verdad:

- `null?`: "¿Es este valor la lista vacía?".
- `eqv?`: "¿Son estos dos valores idénticos?".
- `pair?`: "¿Es este valor un par (un elemento de una lista construida con `cons`)?".
- `symbol?`: "¿Es este valor un símbolo?".
- `number?`: "¿Es este valor un número?".
- `string?`: "¿Es este valor una cadena de texto?".

5 - Predicados

Predicados que no terminan en ?

Los predicados (`=`, `<`, `>`, `<=`, `>=`) son operadores aritméticos de comparación. Por convención, estos operadores no suelen llevar el signo de interrogación en la mayoría de los lenguajes de programación, incluido Racket.

En resumen

La terminación con un signo de interrogación `?` es una convención de estilo en Racket para los predicados, pero no se aplica de manera uniforme a todos ellos. Los operadores de comparación aritmética son una excepción común.

6 - Recursión

La recursión es el mecanismo principal para realizar iteraciones, es decir, para repetir un proceso varias veces. A diferencia de los lenguajes imperativos, que utilizan bucles como `for` y `while`, los lenguajes funcionales utilizan la recursión para lograr el mismo efecto.

6 - Recursión

La recursión es una técnica en la que una función se define en términos de sí misma. En otras palabras, una función recursiva se llama a sí misma dentro de su propio cuerpo. Para que la recursión funcione correctamente, es necesario definir un caso base y un caso recursivo:

- **Caso base:** Es la condición que detiene la recursión. Define el resultado para la entrada más simple posible, sin necesidad de realizar más llamadas recursivas.
- **Caso recursivo:** Es la condición en la que la función se llama a sí misma con una entrada más pequeña o simplificada. La llamada recursiva debe acercar el problema al caso base.

6 - Recursión

Ejemplo en Racket de función recursiva que calcula la suma de los elementos de una lista:

```
(define (suma-lista lista)
```

```
(if (null? lista) ; Caso base: la lista está vacía
```

```
    0 ; La suma de una lista vacía es 0
```

```
    (+ (car lista) ; Caso recursivo: sumar el primer elemento
```

```
        (suma-lista (cdr lista)))) ; y la suma del resto de la lista
```

- El caso base es cuando la lista está vacía (`null? lista`), en cuyo caso la función devuelve 0.
- El caso recursivo es cuando la lista no está vacía. En este caso, la función suma el primer elemento de la lista (`car lista`) con la suma del resto de la lista (`suma-lista (cdr lista)`), que se calcula mediante una llamada recursiva.

6 - Recursión

letrec se utiliza para definir funciones recursivas locales. A diferencia de **let**, que define variables que no pueden referenciarse entre sí durante su inicialización, **letrec** permite que las variables definidas dentro de él se refieran entre sí, lo cual es esencial para definir funciones que se llaman a sí mismas o entre sí.

Funciones Recursivas Locales

El caso de uso más común para **letrec** es la definición de un conjunto de funciones recursivas que son necesarias para la implementación de una función principal, pero que no deben ser visibles fuera de ella.

6 - Recursión

```
(define (mi-funcion lista)
  (letrec ([es-par? (lambda (n)
                      (if (= n 0)
                          #t
                          (es-impar? (sub1 n)))))]
    [es-impar? (lambda (n)
                  (if (= n 0)
                      #f
                      (es-par? (sub1 n))))])
    (filter es-par? lista)))
```

letrec es especialmente útil para definir funciones mutuamente recursivas, es decir, funciones que se llaman entre sí.

las funciones **es-par?** y **es-impar?** se definen dentro de un **letrec**. Estas funciones se llaman entre sí para determinar si un número es par o impar. El uso de **letrec** es crucial aquí, ya que permite que cada función llame a la otra. Si hubiéramos usado **let**, estas funciones no podrían referenciarse entre sí, y el código resultaría en un error.

6 - Recursión

Ámbito Local

Las definiciones realizadas con `letrec` tienen un ámbito local, lo que significa que solo son visibles dentro del `letrec`. Esto es útil para encapsular funciones auxiliares y evitar la contaminación del espacio de nombres global.

En resumen, `letrec` se utiliza para definir funciones recursivas locales, especialmente cuando se necesitan funciones mutuamente recursivas o cuando se desea encapsular funciones auxiliares dentro de un ámbito local.

6 - Recursión

Ventajas de la recursión

- Permite escribir código más conciso y elegante para resolver problemas que involucren repetición.
- Se adapta naturalmente al estilo de programación funcional, ya que no introduce efectos secundarios.
- Facilita el razonamiento sobre el código, ya que cada llamada recursiva resuelve un subproblema más pequeño del problema original.